

DEVOLUTIONS S.A.S.

# Documento de Arquitectura

Bar Inventory APP — Plataforma de Gestión Integral  
Café Colombia Bar · Sedes Galería · Zona T · Modelia

|                              |                                                                                                                         |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Versión del documento</b> | 1.0                                                                                                                     |
| <b>Fecha</b>                 | 30 de septiembre de 2026                                                                                                |
| <b>Proyecto</b>              | App Inventory Bar — Diseño de algoritmos                                                                                |
| <b>Equipo</b>                | Johan Camilo Pedraos · Sofía Pedraza · Jose Florián                                                                     |
| <b>Repositorio</b>           | <a href="https://github.com/johanyopasa03-droid/App-Inventory-bar">github.com/johanyopasa03-droid/App-Inventory-bar</a> |
| <b>Ubicación</b>             | Carpeta /Ejecucion del repositorio documental                                                                           |

# Versión del documento

## Control de versiones

| Versión | Fecha      | Autor               | Descripción del cambio                                                                                                                                                                                                                      |
|---------|------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 30/09/2026 | DevSolutions S.A.S. | Creación del documento: introducción, drivers de arquitectura, decisiones, diagramas de arquitectura, componentes, despliegue, modelo entidad-relación, buenas prácticas, diccionario de términos y matriz de trazabilidad HU → componente. |

## Aprobaciones

| Rol                       | Nombre               | Firma / fecha |
|---------------------------|----------------------|---------------|
| Arquitectura y desarrollo | Johan Camilo Pedraos |               |
| Análisis y base de datos  | Sofía Pedraza        |               |
| Desarrollo y pruebas      | Jose Florián         |               |
| Cliente                   | Café Colombia Bar    |               |

## Documentos relacionados

| Documento                                        | Versión           | Ubicación en el repositorio          |
|--------------------------------------------------|-------------------|--------------------------------------|
| Propuesta Comercial Café Colombia Bar            | V1.1 (03/09/2026) | /Inicio                              |
| Backlog del Producto · Plan de Sprints           | V1.1 (03/09/2026) | /Planeacion                          |
| Historias de Usuario HU01 – HU30                 | V1.0 (03/09/2026) | /Ejecucion/Historias de usuario      |
| Script de base de datos<br>bar_inventory_app.sql | V1.0              | /Ejecucion (anexo de este documento) |

# Tabla de contenido

---

|                                                                  |    |
|------------------------------------------------------------------|----|
| Versión del documento .....                                      | 2  |
| 1. Introducción .....                                            | 4  |
| 1.1 Propósito .....                                              | 4  |
| 1.2 Alcance del sistema .....                                    | 4  |
| 1.3 Audiencia .....                                              | 5  |
| 1.4 Contexto del negocio y actores .....                         | 5  |
| 2. Drivers de arquitectura .....                                 | 6  |
| 2.1 Requisitos no funcionales .....                              | 6  |
| 2.2 Reglas de negocio con impacto arquitectónico .....           | 6  |
| 2.3 Supuestos y decisiones sobre inconsistencias .....           | 7  |
| 3. Diagrama de arquitectura .....                                | 9  |
| 3.1 Estilo arquitectónico .....                                  | 9  |
| 3.2 Stack tecnológico .....                                      | 10 |
| 3.3 Flujo principal: del pedido a la factura .....               | 10 |
| 4. Diagrama de componentes .....                                 | 12 |
| 5. Diagrama de despliegue .....                                  | 14 |
| 5.1 Justificación de VPS Linux + Docker .....                    | 15 |
| 5.2 Operación, respaldos y recuperación .....                    | 15 |
| 6. Modelo entidad-relación .....                                 | 16 |
| 6.1 Descripción de entidades .....                               | 17 |
| 6.2 Reglas de integridad implementadas en la base de datos ..... | 17 |
| 7. Listado de buenas prácticas .....                             | 19 |
| 8. Diccionario de términos .....                                 | 21 |
| Anexo A. Matriz de trazabilidad HU → componente → tablas .....   | 23 |

# 1. Introducción

## 1.1 Propósito

Este documento describe la arquitectura de software de **Bar Inventory APP**, la plataforma web que DevSolutions S.A.S. desarrolla para Café Colombia Bar. Su objetivo es dejar registradas las decisiones técnicas que permiten cumplir las 30 historias de usuario del backlog: cómo se organiza el sistema, qué componentes lo forman, dónde y cómo se despliega, cómo se estructuran los datos y qué prácticas debe seguir el equipo para construirlo y operarlo.

El documento sirve como referencia común para el equipo de desarrollo durante los sprints 3 a 5. También permite al cliente y al docente verificar que la solución propuesta responde a los requisitos funcionales y no funcionales acordados en la Propuesta Comercial V1.1.

## 1.2 Alcance del sistema

Bar Inventory APP centraliza la operación de las tres sedes de la franquicia (Galería, Zona T y Modelia) en una sola aplicación web con acceso diferenciado por rol. El sistema se organiza en 11 módulos funcionales:

| #  | Módulo                     | Historias | Responsabilidad                                                                                  |
|----|----------------------------|-----------|--------------------------------------------------------------------------------------------------|
| 1  | Autenticación y Sesión     | HU01-HU03 | Inicio de sesión, cierre por inactividad (3 min), cierre manual y manejo de pérdida de conexión. |
| 2  | Administración de Sedes    | HU04-HU05 | Crear, editar e inactivar sedes como unidades operativas independientes.                         |
| 3  | Administración de Usuarios | HU06-HU08 | Cuentas con rol y sede asignada; edición e inactivación sin pérdida de historial.                |
| 4  | Catálogo de Productos      | HU09-HU11 | Catálogo global único con precios iguales en todas las sedes.                                    |
| 5  | Gestión de Proveedores     | HU12-HU13 | Directorio informativo de proveedores y su producto asociado.                                    |
| 6  | Gestión de Inventario      | HU14-HU16 | Stock independiente por sede, entradas de mercancía y descuento automático al pedir.             |
| 7  | Gestión de Mesas           | HU17-HU18 | Configuración de mesas por sede y vista de sala en tiempo real.                                  |
| 8  | Gestión de Pedidos         | HU19-HU22 | Apertura, adición de productos, envío inmutable a caja y cola del cajero.                        |
| 9  | Pagos y Facturación        | HU23-HU26 | Pago en efectivo o con tarjeta (datáfono externo), factura automática e historial.               |
| 10 | Reportes                   | HU27-HU29 | Reportes de ventas por rol con filtros combinados y cálculo de ganancia.                         |
| 11 | Auditoría y Trazabilidad   | HU30      | Log de eventos inmutable y consultable por el Administrador.                                     |

**Fuera de alcance** (Propuesta V1.1, sección 3.4), y por tanto sin soporte en la arquitectura: turnos, reservas, combos y descuentos, integración con pasarela de pago, alertas o stock mínimo,

autoregistro, login externo (OAuth/SSO), integración con sistemas legados, anulación de ventas, división de cuenta, cambio de mesa, domicilios, flujo de cocina e inventario por volumen.

### 1.3 Audiencia

- **Equipo de desarrollo:** guía para implementar módulos, contratos de API y modelo de datos.
- **Docente y evaluadores:** evidencia de las decisiones de diseño y su trazabilidad con las historias de usuario.
- **Cliente (Café Colombia Bar):** visión de cómo se despliega y protege la plataforma.

### 1.4 Contexto del negocio y actores

Hoy la franquicia opera sin un sistema centralizado: no hay registro de entradas y salidas de inventario, no se conoce el stock por sede en tiempo real y no hay consolidación de ingresos. Esto genera riesgo de descuadre de caja y pérdida de trazabilidad. La arquitectura responde a esa brecha con tres actores humanos y un actor sistema:

| Actor         | Alcance de datos | Funciones principales                                                                                                                                                                                                        |
|---------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Administrador | Todas las sedes  | Parametriza sedes, usuarios, mesas, productos y proveedores; gestiona inventario de cualquier sede; ve reportes consolidados con ganancia y el log de auditoría. Puede operar como cajero o mesero eligiendo la sede activa. |
| Cajero        | Su sede asignada | Registra entradas de mercancía, consulta stock, atiende la cola de pedidos, procesa pagos, consulta facturas y reportes de su sede (sin costo ni ganancia).                                                                  |
| Mesero        | Su sede asignada | Consulta la vista de sala y el catálogo, abre pedidos en mesas libres, agrega productos y envía el pedido a caja.                                                                                                            |
| Sistema       | —                | Descuenta stock al agregar ítems (HU16), genera la factura al confirmar el pago (HU25) y registra cada evento en el log (HU30).                                                                                              |

## 2. Drivers de arquitectura

Los drivers son los requisitos que más condicionan las decisiones de diseño. Cada uno tiene una respuesta concreta en la arquitectura; si un driver cambia, la decisión asociada debe revisarse.

### 2.1 Requisitos no funcionales

| Atributo             | Meta                                                                   | Respuesta arquitectónica                                                                                                                                                                                                                 |
|----------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rendimiento          | < 2 s por transacción; reportes y log < 2 s (HU29, HU30)               | Índices compuestos sobre (id_sede, estado, fecha); vista v_ventas_detalle para reportes; paginación del lado del servidor; SPA servida como estáticos comprimidos por Nginx.                                                             |
| Concurrencia         | ≥ 20 usuarios simultáneos                                              | Node.js maneja E/S concurrente sin hilos por petición; pool de conexiones a MySQL; transacciones con bloqueo de fila para stock y consecutivo de factura.                                                                                |
| Seguridad            | OWASP Top 10, con evidencia                                            | HTTPS obligatorio, contraseñas con hash (bcrypt/argon2), JWT de corta duración con revocación por sesión, RBAC por rol y sede validado en el servidor, consultas parametrizadas vía ORM, cabeceras de seguridad, rate limiting en login. |
| Inactividad          | 3 minutos exactos, no configurable (HU02)                              | Doble control: temporizador en la SPA y validación de ultima_actividad en el servidor, para que la sesión expire aunque el cliente se manipule.                                                                                          |
| Tiempo real          | Vista de sala, cola del cajero y stock sin recargar (HU15, HU18, HU22) | WebSockets (Socket.IO) con "rooms" por sede; los módulos publican eventos de dominio que el gateway reenvía.                                                                                                                             |
| Conectividad         | Detectar pérdida en ≤ 5 s (HU03)                                       | Heartbeat de Socket.IO más eventos online/offline del navegador; el pedido vive en la base de datos desde cada acción, por lo que no se pierde.                                                                                          |
| Interfaz             | Inglés, responsive, Chrome                                             | SPA React con diseño mobile-first; textos de interfaz en inglés (los nombres de dominio internos se mantienen en español).                                                                                                               |
| Integridad histórica | Sin eliminación de pedidos, ventas, facturas ni auditoría              | Borrado lógico (activo), triggers que bloquean UPDATE/DELETE en factura y log_auditoria, y precios congelados en el detalle del pedido.                                                                                                  |

### 2.2 Reglas de negocio con impacto arquitectónico

| Regla                                               | Dónde se garantiza                                                                                                                        |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Una mesa tiene como máximo un pedido activo (HU19). | Índice único sobre la columna generada pedido.mesa_activa, que la base de datos rechaza aunque dos meseros abran la mesa al mismo tiempo. |

|                                                                                               |                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| El stock se descuenta al agregar el ítem, no al pagar (HU16).                                 | InventarioService dentro de la misma transacción que inserta el detalle, con <code>SELECT ... FOR UPDATE</code> sobre la fila de inventario. El stock es <code>UNSIGNED</code> , por lo que nunca queda negativo. |
| Pedido en <code>EN_CAJA</code> es inmutable (HU21).                                           | Máquina de estados en <code>PedidoService</code> ; los endpoints de edición solo aceptan pedidos ABIERTO.                                                                                                         |
| Cambio de precio no afecta pedidos ya creados (HU10).                                         | <code>detalle_pedido</code> guarda <code>precio_venta_unit</code> y <code>precio_compra_unit</code> al momento del registro.                                                                                      |
| Factura con número secuencial por sede (HU25).                                                | Tabla <code>consecutivo_factura</code> incrementada con bloqueo de fila; único ( <code>id_sede</code> , <code>numero</code> ).                                                                                    |
| Cajero y Mesero solo ven su sede; costo y ganancia solo los ve el Administrador (HU11, HU28). | Middleware RBAC que inyecta el filtro de sede desde el token, nunca desde el parámetro del cliente; los DTO de respuesta omiten <code>valor_compra</code> para roles no administradores.                          |
| Un pago por pedido; no se combinan métodos (HU24).                                            | Único <code>pago.id_pedido</code> .                                                                                                                                                                               |

## 2.3 Supuestos y decisiones sobre inconsistencias

Al revisar los documentos del repositorio se encontraron diferencias entre el **detalle del Backlog (Planeación)** y las **Historias de Usuario individuales (Ejecución)**. Esta arquitectura toma como fuente de verdad las HU individuales y la Propuesta V1.1, porque son las versiones más específicas y coinciden entre sí. Se recomienda actualizar el Backlog para eliminar estas diferencias.

| Tema                         | Backlog (Planeación)                             | HU individual / Propuesta                       | Decisión                                                                                                                                |
|------------------------------|--------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Inactividad (HU02)           | Aviso a los 15 min y cierre 2 min después        | 3 minutos exactos                               | 3 minutos.                                                                                                                              |
| Stock mínimo (HU15)          | Alerta por debajo del mínimo; exportar Excel/PDF | Fuera de alcance; solo se resalta stock en cero | Sin stock mínimo; se resalta stock = 0.                                                                                                 |
| Estados de mesa (HU18)       | Disponible, Ocupada, Pendiente de pago           | LIBRE u OCUPADA                                 | LIBRE / OCUPADA. "Pendiente de pago" se deduce del pedido <code>EN_CAJA</code> .                                                        |
| Factura anulada (HU26)       | Estado Pagada o Anulada                          | Las facturas no se anulan                       | Sin anulación; facturas inmutables.                                                                                                     |
| Entrada de mercancía (HU14)  | Rol Administrador, con costo unitario por ítem   | Cajero (su sede) y Administrador (todas)        | Ambos roles; el proveedor es opcional en la entrada.                                                                                    |
| Cancelación de pedido (HU16) | Cancelación antes del cobro                      | Propuesta: los pedidos no se eliminan           | Estado CANCELADO (solo desde ABIERTO) que reintegra stock y conserva el registro. Debe confirmarse con el cliente quién puede cancelar. |

**Stack no definido en los documentos fuente.** La Propuesta y las HU no fijan lenguajes ni motor de base de datos. El stack de la sección 3.2 es una decisión de este documento, tomada según la

experiencia del equipo y los requisitos de tiempo real. Si el equipo ya trabaja con otro lenguaje de backend, los diagramas siguen siendo válidos: solo cambian los nombres de las tecnologías.

### 3. Diagrama de arquitectura

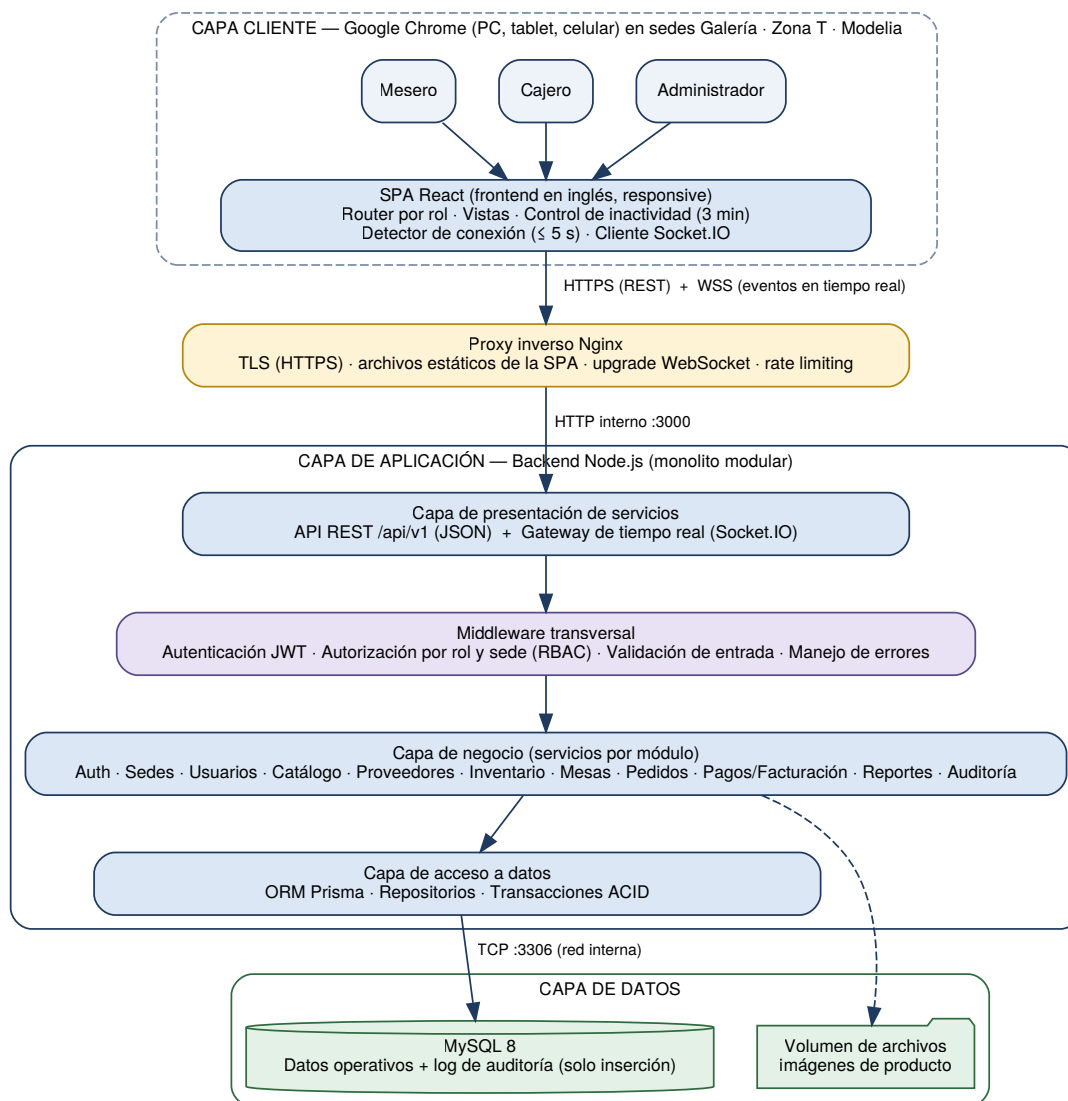


Figura 1. Arquitectura lógica en capas de Bar Inventory APP.

#### 3.1 Estilo arquitectónico

Se adopta una arquitectura **cliente-servidor en tres capas** con un backend **monolito modular**. La SPA en el navegador presenta la información; la API concentra toda la lógica de negocio y la seguridad; MySQL persiste los datos. Dentro del backend, cada módulo funcional (Pedidos, Inventario, Pagos...) tiene su propio controlador, servicio y repositorio, y los módulos se comunican solo a través de sus servicios.

**Por qué no microservicios:** el sistema tiene una sola base de datos con reglas transaccionales que cruzan módulos. Por ejemplo, agregar un ítem toca Pedidos e Inventario en la misma transacción, y pagar toca Pagos, Pedidos, Mesas y Facturación. Separar esto en servicios obligaría a manejar transacciones distribuidas para 20 usuarios y tres sedes, con más infraestructura y más

puntos de falla sin beneficio real. El monolito modular deja la puerta abierta a extraer un módulo (por ejemplo, Reportes) si en el futuro la carga lo justifica.

### 3.2 Stack tecnológico

| Capa            | Tecnología                                                                 | Motivo                                                                                                 |
|-----------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Frontend        | React 18 + TypeScript + Vite                                               | SPA responsive con componentes reutilizables por rol; tipado compartido con el backend.                |
| Tiempo real     | Socket.IO (cliente y servidor)                                             | Reconexión automática y heartbeat (detección de desconexión ≤ 5 s); rooms por sede.                    |
| Backend         | Node.js 20 LTS + Express + TypeScript                                      | Buen manejo de muchas conexiones concurrentes y WebSockets; mismo lenguaje en front y back.            |
| Acceso a datos  | Prisma ORM                                                                 | Consultas parametrizadas (prevención de inyección SQL), migraciones versionadas y transacciones.       |
| Validación      | Zod                                                                        | Validación de entradas en cada endpoint (OWASP A03).                                                   |
| Seguridad       | JWT (cookie httpOnly, SameSite=Strict), bcrypt, Helmet, express-rate-limit | Autenticación sin estado más revocación por tabla session; cabeceras seguras; bloqueo de fuerza bruta. |
| Base de datos   | MySQL 8.0 (InnoDB, utf8mb4)                                                | Transacciones ACID, columnas generadas, CHECK, triggers y JSON; conocido por el equipo.                |
| Servidor web    | Nginx                                                                      | Terminación TLS, estáticos, proxy reverso y upgrade de WebSocket.                                      |
| Infraestructura | VPS Linux + Docker Compose                                                 | Ver sección 5.                                                                                         |
| Calidad         | ESLint, Prettier, Vitest/Jest, Supertest, GitHub Actions                   | Integración continua en cada pull request.                                                             |

### 3.3 Flujo principal: del pedido a la factura

El ciclo de vida del pedido es el corazón del sistema y conecta seis módulos. La Figura 2 muestra sus estados y las historias que gobiernan cada transición.

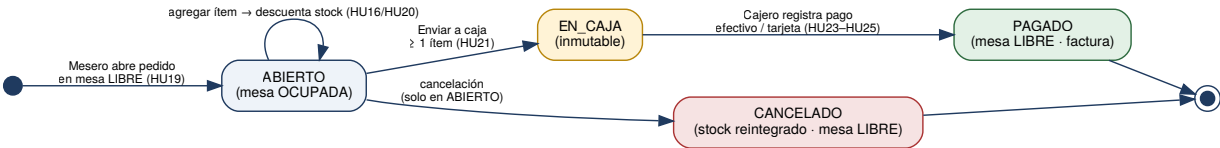


Figura 2. Máquina de estados del pedido y su efecto sobre la mesa y el inventario.

- 1. Abrir pedido:** el mesero elige una mesa LIBRE. En una transacción se crea el pedido ABIERTO y la mesa pasa a OCUPADA; se emite `table.updated` a la sede.
- 2. Agregar ítem:** se bloquea la fila de inventario, se valida stock, se descuenta, se registra el movimiento `SALIDA_PEDIDO` y se recalcula el total. Se emiten `stock.updated` y `order.updated`.
- 3. Enviar a caja:** el pedido pasa a EN\_CAJA y aparece en la cola del cajero (`order.sent_to_cashier`).

4. **Pagar:** se registra el pago, se toma el siguiente consecutivo de la sede, se crea la factura, el pedido pasa a PAGADO y la mesa a LIBRE, todo en una sola transacción. Si un paso falla, no se aplica ninguno.
5. **Auditoría:** cada paso escribe en `log_auditoria` el usuario, la sede, el tipo de evento y la entidad afectada.

## 4. Diagrama de componentes

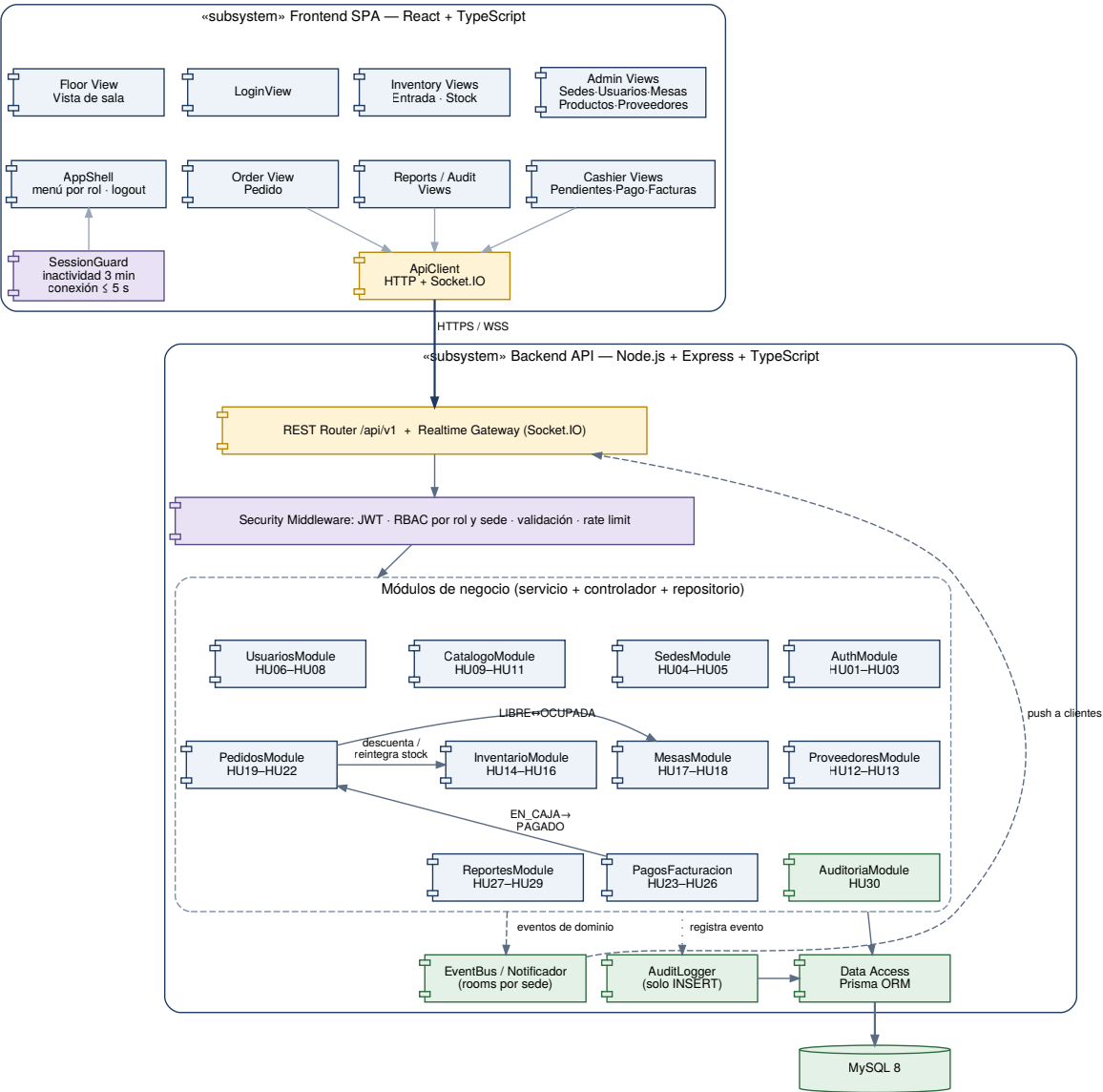


Figura 3. Componentes del frontend y del backend, y sus dependencias.

| Componente                     | Responsabilidad                                                                                               | Interfaz que expone                                   |
|--------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| SessionGuard (SPA)             | Temporizador de inactividad de 3 min, detección de conexión y redirección al login.                           | Hook <code>useSession()</code>                        |
| ApiClient (SPA)                | Única puerta de la SPA hacia el servidor: peticiones HTTP, manejo de 401/403 y suscripción a eventos.         | <code>api.get/post...</code> , <code>socket.on</code> |
| REST Router + Realtime Gateway | Expone <code>/api/v1/*</code> y <code>/socket.io</code> ; une a cada socket al room de su sede.               | HTTP JSON, eventos WS                                 |
| Security Middleware            | Verifica JWT y sesión vigente, aplica permisos por rol, fija el alcance de sede y valida esquemas de entrada. | Middleware Express                                    |

|                               |                                                                                                              |                                     |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------|
| AuthModule                    | Login, logout, bloqueo por intentos, renovación de actividad y cierre de sesiones revocadas.                 | /auth/login, /auth/logout, /auth/me |
| Sedes / Usuarios / Mesas      | Parametrización y borrado lógico; validaciones de unicidad y de impacto (sede con pedidos abiertos).         | /sedes, /usuarios, /mesas           |
| Catálogo / Proveedores        | Productos, categorías, imágenes y asociación con proveedor.                                                  | /productos, /proveedores            |
| InventarioModule              | Entradas, consulta de stock, descuento y reintegro transaccional.                                            | /inventario + servicio interno      |
| PedidosModule                 | Máquina de estados del pedido y cola de caja.                                                                | /pedidos                            |
| PagosFacturacion              | Pago, cálculo de cambio, consecutivo y factura imprimible.                                                   | /pagos, /facturas                   |
| ReportesModule                | Reportes por rol con filtros AND y exportación.                                                              | /reportes/ventas                    |
| AuditoriaModule + AuditLogger | El logger lo usan todos los módulos para escribir; el módulo permite al Administrador consultar con filtros. | /auditoria                          |
| EventBus                      | Desacopla los módulos del transporte WebSocket: el módulo publica y el gateway entrega.                      | events.emit(sede, tipo, payload)    |

## 5. Diagrama de despliegue

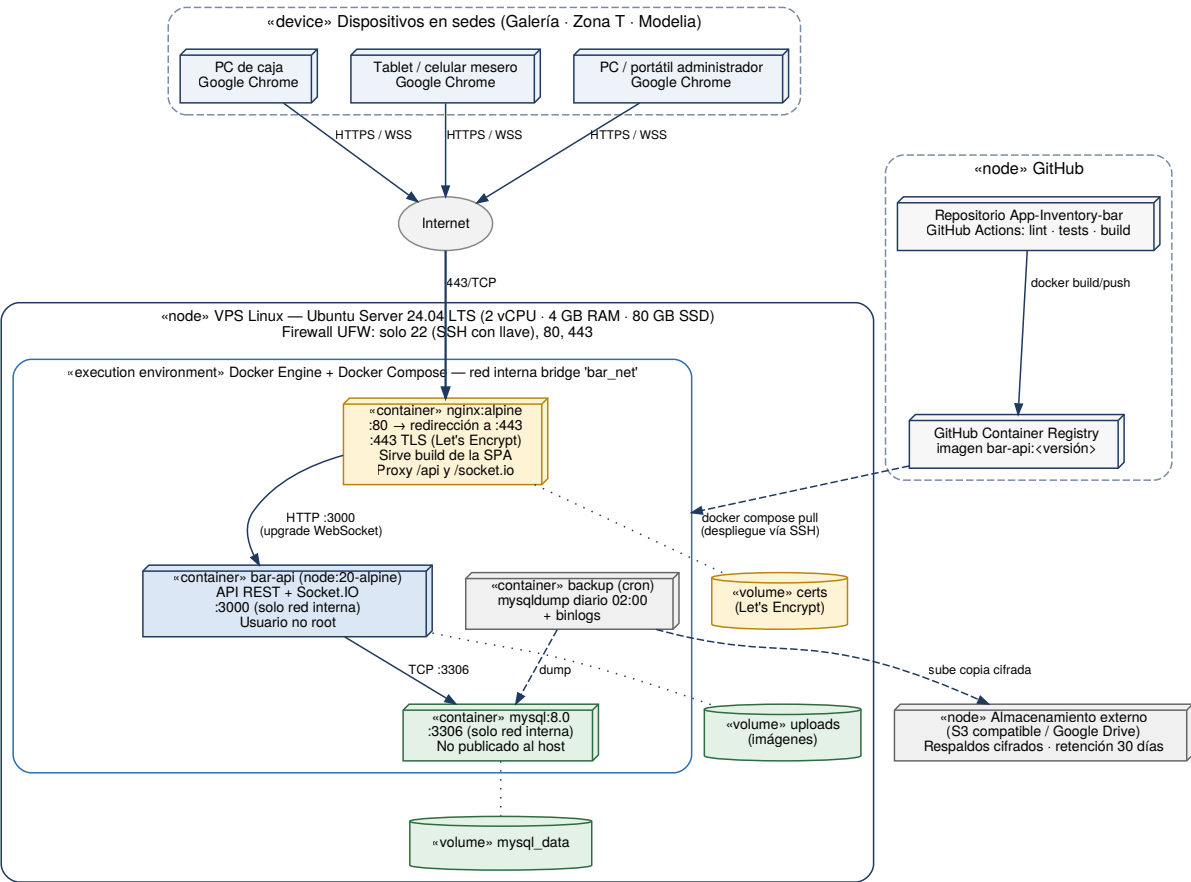


Figura 4. Despliegue en un VPS Linux con contenedores Docker orquestados por Docker Compose.

| Nodo / contenedor    | Puertos             | Configuración clave                                                                                                                                                 |
|----------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VPS Ubuntu 24.04 LTS | 22, 80, 443         | 2 vCPU, 4 GB RAM, 80 GB SSD (suficiente para 20 usuarios concurrentes). UFW activo, SSH solo con llave, fail2ban, actualizaciones de seguridad automáticas.         |
| nginx                | 80 → 443 (públicos) | TLS 1.2+ con Let's Encrypt y renovación automática; HSTS; gzip; proxy_set_header Upgrade para /socket.io; límite de peticiones a /api/v1/auth/login.                |
| bar-api              | 3000 (interno)      | Imagen inmutable versionada; usuario no root; variables de entorno desde archivo .env fuera del repositorio; health check / api/v1/health; restart: unless-stopped. |
| mysql                | 3306 (interno)      | Sin puerto publicado al host; usuario de aplicación con privilegios mínimos (sin DROP ni GRANT); binlog activo para recuperación a un punto en el tiempo.           |
| backup               | —                   | mysqldump --single-transaction diario a las 02:00, comprimido y cifrado, subido a almacenamiento externo con retención de 30 días.                                  |

## 5.1 Justificación de VPS Linux + Docker

Para el tamaño de la operación (3 sedes, unos 20 usuarios concurrentes, un solo cliente) un VPS con Docker Compose es la opción con mejor relación entre costo, control y simplicidad. Kubernetes o un PaaS añadirían costo y complejidad que este volumen no necesita. Docker garantiza que el entorno de desarrollo, pruebas y producción sea el mismo, y permite desplegar una versión nueva o volver a la anterior cambiando la etiqueta de la imagen.

La decisión tiene un riesgo aceptado: **el VPS es un punto único de falla**. Si el servidor cae, las tres sedes quedan sin sistema. Se mitiga con:

- **Contenedores separados** (proxy, API, base de datos) en lugar de un solo contenedor, y datos en volúmenes persistentes, para que recrear la aplicación nunca borre información.
- **Respaldos fuera del VPS** con prueba de restauración mensual.
- **Monitoreo de disponibilidad** externo (por ejemplo, UptimeRobot sobre `/health`) con alerta al equipo.
- Un procedimiento de recuperación documentado, con las metas de la sección 5.2.

## 5.2 Operación, respaldos y recuperación

| Aspecto                       | Definición                                                                                                                                                                                                                                                                 |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Despliegue                    | Merge a main → GitHub Actions ejecuta lint y pruebas → construye y publica <code>bar-api:&lt;versión&gt;</code> → en el VPS:<br><code>docker compose pull &amp;&amp; docker compose up -d</code> . Las migraciones de Prisma se aplican antes de iniciar la nueva versión. |
| Rollback                      | Volver a la etiqueta de imagen anterior y ejecutar <code>docker compose up -d</code> . Las migraciones deben ser compatibles hacia atrás.                                                                                                                                  |
| RPO (pérdida máxima de datos) | ≤ 24 h con el dump diario; ≤ 1 h si se respaldan también los binlogs cada hora.                                                                                                                                                                                            |
| RTO (tiempo de recuperación)  | ≤ 4 h: aprovisionar un VPS nuevo, restaurar el último respaldo y levantar el <code>docker-compose.yml</code> versionado.                                                                                                                                                   |
| Logs                          | Logs de contenedor en JSON con rotación ( <code>max-size</code> ); son distintos del log de auditoría de negocio, que vive en MySQL.                                                                                                                                       |

## 6. Modelo entidad-relación

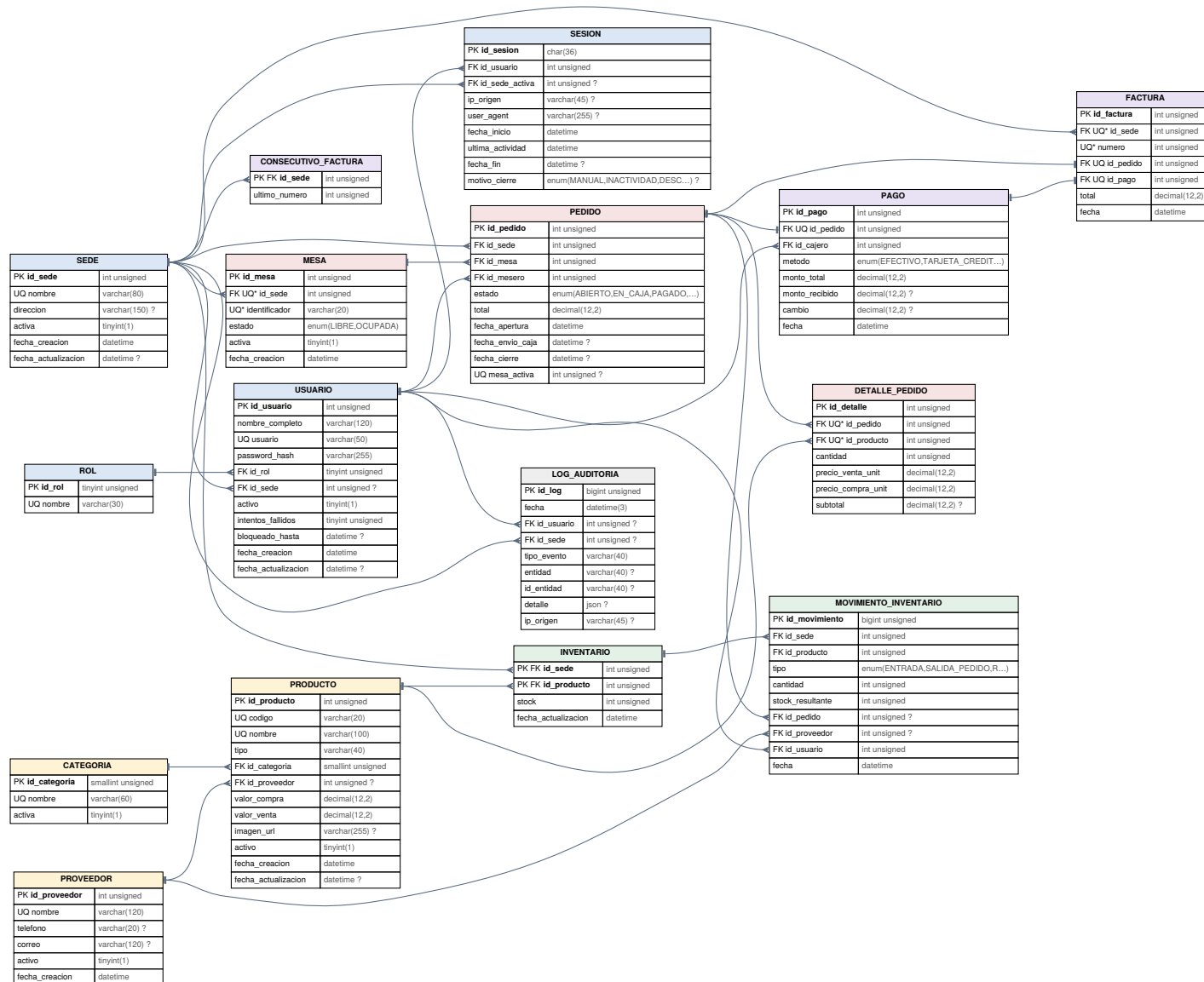


Figura 5. Modelo entidad-relación (generado del script `bar_inventory_app.sql` ejecutado en MySQL 8). PK = clave primaria · FK = clave foránea · UQ = único · UQ\* = parte de un único compuesto · ? = admite nulo. Pata de gallo = "muchos"; barra = "uno".

## 6.1 Descripción de entidades

| Entidad               | Descripción y relaciones                                                                                                                                                        |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SEDE                  | Unidad operativa independiente. Tiene muchos usuarios, mesas, registros de inventario, pedidos y facturas.                                                                      |
| ROL                   | Catálogo fijo: ADMINISTRADOR, CAJERO, MESERO. Cada usuario tiene exactamente un rol.                                                                                            |
| USUARIO               | Persona que opera el sistema. Pertenece a un rol y a una sede (el Administrador puede no tener sede fija). Guarda el hash de la contraseña y el control de intentos fallidos.   |
| SESION                | Cada inicio de sesión. Permite cerrar por inactividad, revocar al inactivar un usuario (HU08) y registrar el motivo del cierre (HU03). Guarda la sede activa del Administrador. |
| CATEGORIA             | Agrupar productos para filtros de catálogo y reportes.                                                                                                                          |
| PROVEEDOR             | Directorio informativo. Un proveedor suministra muchos productos; cada producto tiene un proveedor principal.                                                                   |
| PRODUCTO              | Catálogo global con código, valor de compra y de venta; precio igual en todas las sedes.                                                                                        |
| INVENTARIO            | Entidad asociativa SEDE × PRODUCTO con el stock actual en unidades enteras.                                                                                                     |
| MOVIMIENTO_INVENTARIO | Historial de entradas, salidas por pedido y reintegros, con el stock resultante. Permite reconstruir y auditar el inventario.                                                   |
| MESA                  | Mesa de una sede con estado LIBRE/OCUPADA. El identificador es único dentro de la sede.                                                                                         |
| PEDIDO                | Orden de una mesa creada por un mesero, con estados ABIERTO, EN_CAJA, PAGADO y CANCELADO.                                                                                       |
| DETALLE_PEDIDO        | Línea de pedido con la cantidad y los precios congelados al momento del registro.                                                                                               |
| PAGO                  | Pago único del pedido, hecho por un cajero, con método, monto recibido y cambio.                                                                                                |
| CONSECUTIVO_FACTURA   | Último número de factura emitido por cada sede.                                                                                                                                 |
| FACTURA               | Comprobante inmutable 1:1 con el pedido y con el pago.                                                                                                                          |
| LOG_AUDITORIA         | Registro de eventos de solo inserción, con detalle JSON (valor anterior y nuevo).                                                                                               |

## 6.2 Reglas de integridad implementadas en la base de datos

Las reglas críticas no dependen solo del código de la aplicación: la base de datos las hace cumplir aunque haya un error en el backend o un acceso directo.

- `uq_pedido_mesa_activa`: impide dos pedidos activos en la misma mesa.
- `inventario.stock INT UNSIGNED y CHECK (cantidad > 0)`: no hay stock negativo ni cantidades en cero.
- `ck_pago_efectivo`: en efectivo, el monto recibido debe cubrir el total.
- `uq_factura_sede_numero`: consecutivo único por sede.
- Triggers `trg_log_no_update/delete` y `trg_factura_no_update/delete`: auditoría y facturas inmutables.

- Únicos de negocio: nombre de sede, usuario, nombre y código de producto, nombre de proveedor y (`id_sede`, `identificador`) de mesa.

El script completo (tablas, índices, triggers, vista de reportes y datos iniciales de roles y sedes) se entrega como anexo en `bar_inventory_app.sql`. Se probó en MySQL 8.0 verificando que rechaza un segundo pedido activo en la misma mesa y cualquier modificación del log de auditoría.

## 7. Listado de buenas prácticas

### 7.1 Seguridad (OWASP Top 10)

| #   | Práctica                              | Aplicación en el proyecto                                                                                                                 | OWASP |
|-----|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-------|
| S1  | Autorización en el servidor           | Cada endpoint declara los roles permitidos; la sede se toma del token, nunca del cuerpo de la petición.                                   | A01   |
| S2  | Contraseñas con hash                  | bcrypt con costo $\geq 12$ ; nunca en texto plano ni en logs.                                                                             | A02   |
| S3  | Cifrado en tránsito                   | Solo HTTPS/WSS, HSTS, TLS 1.2 o superior.                                                                                                 | A02   |
| S4  | Consultas parametrizadas y validación | Prisma + Zod en cada entrada; nada de SQL concatenado.                                                                                    | A03   |
| S5  | Protección contra fuerza bruta        | Bloqueo temporal tras intentos fallidos y rate limit en login; mensaje de error genérico.                                                 | A07   |
| S6  | Sesiones seguras                      | JWT en cookie httpOnly, Secure y SameSite=Strict; expiración por inactividad validada en el servidor; revocación al inactivar el usuario. | A07   |
| S7  | Cabeceras y configuración segura      | Helmet (CSP, X-Frame-Options), base de datos sin puerto público, errores sin stack trace en producción.                                   | A05   |
| S8  | Dependencias al día                   | npm audit y Dependabot en el repositorio.                                                                                                 | A06   |
| S9  | Registro y monitoreo                  | Log de auditoría inmutable de logins, cambios y pagos.                                                                                    | A09   |
| S10 | Secretos fuera del código             | Variables de entorno; .env en .gitignore; contraseñas distintas por ambiente.                                                             | A05   |
| S11 | Carga segura de imágenes              | Validar tipo MIME y tamaño ( $\leq 2$ MB), renombrar con UUID y servir desde un volumen fuera del código.                                 | A04   |

### 7.2 Código y diseño

- **Separación en capas:** controlador (HTTP) → servicio (reglas de negocio) → repositorio (datos). La lógica de negocio nunca va en el controlador ni en el frontend.
- **Principios SOLID y DRY:** un servicio por agregado; validaciones y permisos centralizados en middleware reusable.
- **Transacciones explícitas** en toda operación que toca varias tablas (agregar ítem, pagar, inactivar sede).
- **Convenciones de nombres:** tablas y columnas en `snake_case` y en español; código en `camelCase`; textos de interfaz en inglés mediante un archivo de recursos.
- **API REST consistente:** versionado `/api/v1`, sustantivos en plural, códigos HTTP correctos (201, 400, 401, 403, 404, 409) y formato de error uniforme `{code, message}`.
- **Dinero en DECIMAL**, nunca en punto flotante.
- **Borrado lógico** (`activo = 0`) para sedes, usuarios, productos, proveedores y mesas.

### 7.3 Calidad y pruebas

- Pruebas unitarias de servicios críticos (stock, máquina de estados del pedido, cálculo de cambio) con cobertura  $\geq 70$  %.
- Pruebas de integración de la API con Supertest contra una base MySQL de prueba en contenedor.

- Prueba de concurrencia: dos meseros abriendo la misma mesa y dos pedidos consumiendo la última unidad.
- Prueba de carga con 20 usuarios simultáneos (k6 o JMeter) para evidenciar el tiempo de respuesta menor a 2 s.
- Criterios de aceptación de cada HU convertidos en casos de prueba.

#### 7.4 Gestión de configuración y proceso

- Flujo GitHub Flow: rama por HU ( `feature/HU19-crear-pedido` ), pull request con revisión de al menos un compañero y CI en verde antes del merge.
- Commits convencionales ( `feat:` , `fix:` , `docs:` ) y versionado semántico de las imágenes.
- Migraciones de base de datos versionadas en el repositorio; prohibido modificar el esquema a mano en producción.
- Repositorio documental organizado por fase (Inicio, Planeación, Ejecución, Monitoreo, Cierre), con este documento en la carpeta Ejecución.
- Definición de Hecho (DoD): código revisado, pruebas en verde, criterios de aceptación verificados y documentación actualizada.

#### 7.5 Operación

- RespalDOS diarios automáticos fuera del VPS, con prueba de restauración mensual.
- Health checks y reinicio automático de contenedores.
- Monitoreo externo de disponibilidad con alerta al equipo.
- Actualizaciones de seguridad del sistema operativo y de las imágenes base cada mes.

## 8. Diccionario de términos

| Término                    | Definición                                                                                                                                  |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| ACID                       | Propiedades de una transacción de base de datos (Atomicidad, Consistencia, Aislamiento, Durabilidad): o se aplica completa o no se aplica.  |
| Administrador              | Rol con acceso a todas las sedes, parametrización del sistema, reportes con ganancia y auditoría.                                           |
| API REST                   | Interfaz de servicios web que expone recursos mediante URL y métodos HTTP (GET, POST, PUT, PATCH).                                          |
| Auditoría (log de eventos) | Registro inmutable de acciones relevantes: quién, qué, sobre qué entidad, en qué sede y cuándo.                                             |
| Borrado lógico             | Marcar un registro como inactivo en lugar de eliminarlo, para conservar el historial.                                                       |
| Cajero                     | Rol que procesa pagos, gestiona el inventario y consulta facturas y reportes de su sede.                                                    |
| Catálogo global            | Lista única de productos compartida por todas las sedes, con el mismo precio.                                                               |
| Consecutivo de factura     | Número secuencial sin saltos ni repeticiones, independiente por sede.                                                                       |
| Contenedor (Docker)        | Paquete aislado que incluye una aplicación y sus dependencias, y se ejecuta igual en cualquier servidor.                                    |
| Datáfono                   | Terminal externa de pago con tarjeta; el sistema solo registra que el cobro fue aprobado.                                                   |
| Docker Compose             | Herramienta que define y levanta varios contenedores relacionados con un archivo <code>docker-compose.yml</code> .                          |
| DTO                        | Objeto de transferencia de datos: la forma exacta de la información que entra o sale de la API.                                             |
| Entrada de mercancía       | Registro que suma unidades al stock de un producto en una sede.                                                                             |
| Estado de mesa             | LIBRE (sin pedido activo) u OCUPADA (con un pedido ABIERTO o EN_CAJA).                                                                      |
| Estado de pedido           | ABIERTO (editable), EN_CAJA (enviado, inmutable), PAGADO (cobrado y facturado), CANCELADO (anulado antes del envío, con stock reintegrado). |
| Factura                    | Comprobante inmutable generado automáticamente al registrar el pago.                                                                        |
| Ganancia                   | Valor de venta menos valor de compra de los productos vendidos; solo visible para el Administrador.                                         |
| HU (Historia de Usuario)   | Requisito escrito desde la perspectiva del usuario ("Como... quiero... para...") con criterios de aceptación.                               |
| Inactividad                | Periodo sin interacción del usuario; a los 3 minutos la sesión se cierra.                                                                   |
| JWT                        | JSON Web Token: credencial firmada que identifica al usuario en cada petición.                                                              |
| Mesero                     | Rol que abre pedidos en mesas de su sede, agrega productos y los envía a caja.                                                              |
| Monolito modular           | Aplicación desplegada como una sola unidad pero organizada internamente en módulos independientes.                                          |
| Movimiento de inventario   | Cada cambio de stock (entrada, salida por pedido, reintegro) con su referencia y el stock resultante.                                       |

|                   |                                                                                                                              |
|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| Nginx             | Servidor web y proxy inverso que recibe el tráfico HTTPS y lo dirige a la API.                                               |
| ORM               | Mapeo objeto-relacional: biblioteca que traduce operaciones del código a SQL seguro.                                         |
| OWASP Top 10      | Lista de los diez riesgos de seguridad más críticos en aplicaciones web.                                                     |
| Pedido            | Orden de consumo asociada a una mesa y a un mesero.                                                                          |
| Proveedor         | Empresa o persona que suministra productos; registro informativo.                                                            |
| RBAC              | Control de acceso basado en roles: los permisos dependen del rol del usuario.                                                |
| Responsive        | Interfaz que se adapta al tamaño de pantalla (PC, tablet, celular).                                                          |
| RPO / RTO         | Pérdida máxima de datos tolerable / tiempo máximo para restablecer el servicio tras una falla.                               |
| Sede              | Punto de operación de la franquicia (Galería, Zona T, Modelia) con mesas, inventario y usuarios propios.                     |
| Sesión            | Periodo autenticado de un usuario, desde el login hasta el cierre manual, por inactividad, por desconexión o por revocación. |
| SPA               | Single Page Application: aplicación web que carga una sola vez y actualiza la vista sin recargar la página.                  |
| Sprint            | Iteración de Scrum de dos semanas que entrega un incremento funcional.                                                       |
| Stock             | Cantidad disponible de un producto en una sede, en unidades enteras.                                                         |
| Story Points (SP) | Unidad relativa de estimación del esfuerzo de una HU (escala de Fibonacci).                                                  |
| TLS / HTTPS       | Protocolo que cifra la comunicación entre el navegador y el servidor.                                                        |
| Trigger           | Procedimiento de la base de datos que se ejecuta automáticamente ante un INSERT, UPDATE o DELETE.                            |
| VPS               | Servidor privado virtual: máquina virtual en la nube con sistema operativo propio (aquí, Linux).                             |
| WebSocket         | Canal de comunicación bidireccional y permanente que permite al servidor enviar actualizaciones en tiempo real.              |

## Anexo A. Matriz de trazabilidad HU → componente → tablas

| HU      | Historia                                       | Componente                       | Tablas principales                                |
|---------|------------------------------------------------|----------------------------------|---------------------------------------------------|
| HU01    | Inicio de sesión con credenciales              | AuthModule                       | usuario, sesion, log_auditoria                    |
| HU02    | Cierre automático por inactividad              | SessionGuard, AuthModule         | sesion                                            |
| HU03    | Cierre manual y pérdida de conexión            | SessionGuard, Gateway            | sesion, log_auditoria                             |
| HU04-05 | Crear, editar e inactivar sede                 | SedesModule                      | sede, consecutivo_factura                         |
| HU06-08 | Crear, editar e inactivar usuario              | UsuariosModule                   | usuario, rol, sesion                              |
| HU09-11 | Catálogo de productos                          | CatalogoModule                   | producto, categoria                               |
| HU12-13 | Proveedores y asociación                       | ProveedoresModule                | proveedor, producto                               |
| HU14    | Entrada de mercancía                           | InventarioModule                 | inventario, movimiento_inventario                 |
| HU15    | Stock por sede                                 | InventarioModule, EventBus       | inventario                                        |
| HU16    | Descuento automático de stock                  | PedidosModule → InventarioModule | inventario, movimiento_inventario, detalle_pedido |
| HU17-18 | Mesas y vista de sala                          | MesasModule, EventBus            | mesa, pedido                                      |
| HU19-21 | Crear pedido, agregar productos, enviar a caja | PedidosModule                    | pedido, detalle_pedido, mesa                      |
| HU22    | Pedidos activos del cajero                     | PedidosModule, EventBus          | pedido                                            |
| HU23-24 | Pago en efectivo / tarjeta                     | PagosFacturacion                 | pago, pedido, mesa                                |
| HU25-26 | Generar factura e historial                    | PagosFacturacion                 | factura, consecutivo_factura                      |
| HU27-29 | Reportes de ventas y filtros                   | ReportesModule                   | v_ventas_detalle                                  |
| HU30    | Log de eventos                                 | AuditoriaModule, AuditLogger     | log_auditoria                                     |